Practical No.1 : Simple Reflex Agent

```
class Main {
   public static void main(String[] args) {
      char location = 'A';
      String A = "Dirty", B = "Dirty";

      for (int i = 0; i < 4; i++) {
         String status = (location == 'A') ? A : B;

         if (status.equals("Dirty")) {
            System.out.println(location + " -> Suck");

            if (location == 'A')
               A = "Clean";
            else
               B = "Clean";
         }
         else if (location == 'A') {
            System.out.println("A -> Right");
            location = 'B';
         }
         else {
            System.out.println("B -> Left");
            location = 'A';
         }
      }
   }
}
```

Practical No. 2 : A* Algorithm

```
import java.util.*;

class Main {
   static int h(int[] a, int[] goal) {
      int d = 0;
      for (int i = 0; i < 9; i++)
         if (a[i] != 0 && a[i] != goal[i]) d++;
      return d;
   }

   public static void main(String[] args) {
      int[] start = {1, 2, 3, 4, 0, 6, 7, 5, 8};
      int[] goal  = {1, 2, 3, 4, 5, 6, 7, 8, 0};

      PriorityQueue<int[]> q = new PriorityQueue<>(
         (a, b) -> (a[9] + a[10]) - (b[9] + b[10])
      );

      int[] s = Arrays.copyOf(start, 11);
      s[9] = 0;
      s[10] = h(start, goal);
      q.add(s);
```

```
        Set<String> visited = new HashSet<>();

        while (!q.isEmpty()) {
            int[] cur = q.poll();
            int[] board = Arrays.copyOf(cur, 9);

            if (Arrays.equals(board, goal)) {
                System.out.println("Goal reached!");
                return;
            }

            if (!visited.add(Arrays.toString(board))) continue;

            int z = 0;
            while (board[z] != 0) z++;

            int[] moves = {-3, 3, -1, 1};

            for (int m : moves) {
                int n = z + m;
                if (n < 0 || n > 8) continue;
                if (Math.abs(m) == 1 && z / 3 != n / 3) continue;

                int[] next = board.clone();
                next[z] = next[n];
                next[n] = 0;

                if (!visited.contains(Arrays.toString(next))) {
                    int[] node = Arrays.copyOf(next, 11);
                    node[9] = cur[9] + 1;
                    node[10] = h(next, goal);
                    q.add(node);
                }
            }
        }

        System.out.println("No solution!");
    }
}
```

Practical No.3 : Tower of Honoi

```
class Main {
    static void hanoi(int n, char source, char aux, char dest) {
        if (n == 1) {
            System.out.println("Move disk 1 from " + source + " to " + dest);
```

```
            return;
        }

        hanoi(n - 1, source, dest, aux);
        System.out.println("Move disk " + n + " from " + source + " to " + dest);
        hanoi(n - 1, aux, source, dest);
    }

    public static void main(String[] args) {
        int n = 3;
        hanoi(n, 'A', 'B', 'C');
    }
}
```

Practical No.4 : Local Beam Search

```
import java.util.*;

class Main {
    static int n = 4, k = 2;
    static int[][] d = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    static int cost(int[] p) {
        int c = 0;
        for (int i = 0; i < n - 1; i++)
            c += d[p[i]][p[i + 1]];
        return c + d[p[n - 1]][p[0]];
    }

    public static void main(String[] args) {
        int[] p = {0, 1, 2, 3};
        int best = cost(p);

        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int[] q = p.clone();
                int t = q[i]; q[i] = q[j]; q[j] = t;

                if (cost(q) < best) {
                    best = cost(q);
                    p = q;
                }
            }
        }

        System.out.println("Best Cost: " + best);
        System.out.println("Best Tour: " + Arrays.toString(p));
    }
}
```

Practical No. 5 : BFS for Robot Path Planning

```
import java.util.*;

class Main {
   static int[][] grid = {
      {0, 0, 0},
      {1, 1, 0},
      {0, 0, 0}
   };

   public static void main(String[] args) {
      Queue<int[]> q = new LinkedList<>();
      boolean[][] visited = new boolean[3][3];
      int[] dx = {0, 0, 1, -1};
      int[] dy = {1, -1, 0, 0};

      q.add(new int[]{0, 0, 0});
      visited[0][0] = true;

      while (!q.isEmpty()) {
         int[] p = q.poll();
         int x = p[0], y = p[1], d = p[2];

         if (x == 2 && y == 2) {
            System.out.println("Shortest path: " + d);
            return;
         }

         for (int i = 0; i < 4; i++) {
            int a = x + dx[i], b = y + dy[i];

            if (a >= 0 && a < 3 && b >= 0 && b < 3
               && grid[a][b] == 0 && !visited[a][b]) {
               visited[a][b] = true;
               q.add(new int[]{a, b, d + 1});
            }
         }
      }

      System.out.println("No path found");   }}
```

Practical No.6 : DFS for Maze Solving

```
class Main {
   static int[][] maze = {
      {0, 0, 1},
      {1, 0, 1},
      {0, 0, 0}
   };

   static boolean dfs(int x, int y) {
      if (x < 0 || y < 0 || x >= 3 || y >= 3
         || maze[x][y] != 0)
```

```
            return false;

        if (x == 2 && y == 2) return true;

        maze[x][y] = 2;

        if (dfs(x + 1, y) || dfs(x, y + 1) ||
            dfs(x - 1, y) || dfs(x, y - 1)) {
            System.out.println(x + " " + y);
            return true;
        }

        return false;
    }

    public static void main(String[] args) {
        if (dfs(0, 0))
            System.out.println("Path found!");
        else
            System.out.println("No path found!");
    }
}
```

Practical No. 7 : 8 Queen Problem

```
class Main {
    static int N = 8;
    static int[] board = new int[N];

    static boolean safe(int r, int c) {
        for (int i = 0; i < r; i++)
            if (board[i] == c ||
                Math.abs(board[i] - c) == Math.abs(i - r))
                return false;
        return true;
    }

    static boolean solve(int r) {
        if (r == N) return true;

        for (int c = 0; c < N; c++) {
            if (safe(r, c)) {
                board[r] = c;
                if (solve(r + 1)) return true;
            }
        }
        return false;
    }

    public static void main(String[] args) {
        solve(0);

        for (int r = 0; r < N; r++) {
            for (int c = 0; c < N; c++)
                System.out.print(board[r] == c ? "Q " : ". ");
```

```
            System.out.println();
        }
    }
}
```

Practical No. 8: AI Chatbot

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("You: ");
            String s = sc.nextLine().toLowerCase();

            if (s.contains("hello")) {
                System.out.println("Bot: Hello!");
            }
            else if (s.contains("name")) {
                System.out.println("Bot: I am a chatbot.");
            }
            else if (s.contains("how are you")) {
                System.out.println("Bot: I am fine!");
            }
            else if (s.contains("bye")) {
                System.out.println("Bot: Goodbye!");
                break;
            }
            else {
                System.out.println("Bot: Sorry, I don't understand.");
            }
        }

        sc.close();    }}
```